

Master en Inteligencia Artificial, Reconocimiento de Formas e
Imagen Digital

Primer boceto de memoria tecnica

Sistemas recomendadores de peliculas

Alumno: Juan Castello Beltran

Fecha: 4 de mayo de 2026

Índice

1. Objetivo y enfoque de la memoria	3
2. Datos, preprocesado y organizacion del sistema	3
2.1. Fuentes de datos de partida	3
2.2. Limpieza y estructuracion inicial del catalogo	4
2.3. Separacion entre entrenamiento y test	5
2.4. Enriquecimiento del catalogo con TMDb	5
2.5. Inicializacion y gestion del registro de usuarios	6
2.6. Creacion de nuevos usuarios y actualizacion del perfil	7
2.7. Papel de esta fase en el resto del sistema	8
3. Sistema recomendador basado en contenido	8
3.1. Que informacion utiliza	8
3.2. Como se representa el usuario	9
3.3. Como se representan las peliculas	9
3.4. Calculo del score de recomendacion	9
3.5. Decisiones de implementacion que interesa justificar	10
3.6. Limitaciones a comentar	10
4. Sistema recomendador colaborativo	10
4.1. Datos de entrada y papel de cada fuente	10
4.2. Representacion del usuario	11
4.3. Construcccion de la matriz usuario-genero	11
4.4. Calculo de similitud entre usuarios	12
4.5. Obtencion y persistencia de vecinos	13
4.6. Generacion de peliculas candidatas	13
4.7. Prediccion del rating de una pelicula	14
4.8. Confianza y score final de ranking	14
4.9. Ordenacion final y control de soporte	15
4.10. Casos de usuarios nuevos y perfiles incompletos	15
4.11. Decisiones de implementacion que interesa destacar	15
4.12. Limitaciones a comentar	16
5. Sistema recomendador hibrido	16
5.1. Idea general	16
5.2. Normalizacion de puntuaciones	17
5.3. Pesos dinamicos	17
5.4. Que debe explicarse en la mezcla final	17

5.5. Decisiones de implementacion que interesa justificar	18
5.6. Limitaciones a comentar	18
6. Sistema recomendador para grupos	18
6.1. Enfoque propuesto	18
6.2. Que debe explicarse en esta seccion	19
6.3. Decision de diseno pendiente	19
7. Interfaz y flujo de uso	19
7.1. Que conviene explicar	19
8. Evaluacion del sistema	20
8.1. Protocolo de evaluacion que debe describirse	20
8.2. Metricas que interesa incluir	20
8.3. Que debe contarse en la memoria	20
9. Conclusiones y trabajo pendiente	20

1. Objetivo y enfoque de la memoria

Esta memoria debe ir directa al grano: explicar como se ha implementado cada sistema recomendador, que informacion utiliza, que decisiones de disenio se han tomado y que limitaciones presenta cada enfoque. No hace falta dedicar mucho espacio a teoria general; lo importante es describir el comportamiento real del proyecto.

En esta primera version conviene dejar claro desde el principio que el trabajo cubre cuatro bloques:

- Sistema recomendador basado en contenido.
- Sistema recomendador colaborativo.
- Sistema recomendador hibrido.
- Sistema recomendador para grupos, incluido como disenio previsto aunque todavia no este implementado.

Tambien es recomendable indicar que la memoria se centra en dos preguntas:

- Como se genera la recomendacion en cada caso.
- Por que se han elegido esas tecnicas y no otras.

2. Datos, preprocesado y organizacion del sistema

Antes de entrenar y utilizar los recomendadores fue necesario construir una capa previa de datos consistente. Esta fase de preprocesado no se limita a limpiar ficheros, sino que define que informacion utilizará cada algoritmo, como se representan las peliculas y de que forma se construye el perfil persistente de los usuarios. En nuestro caso, esta parte tiene ademas un elemento diferencial: el enriquecimiento del catalogo de peliculas con metadatos adicionales obtenidos mediante llamadas a la API de TMDb.

2.1. Fuentes de datos de partida

El sistema parte de varios ficheros crudos ubicados en `Data/Raw_data/`. Los mas importantes son:

- `peliculas.csv`, que contiene la informacion base del catalogo.
- `ratings_small.csv`, con las valoraciones de los usuarios.
- `keywords.csv`, que aporta palabras clave por pelicula.

- `generos.csv`, utilizado para traducir los identificadores numericos de genero a nombres legibles.
- `links.csv`, cargado como parte del pipeline original.

Durante esta fase se generan varios artefactos intermedios y finales en la carpeta de datos procesados. Los mas relevantes para el resto del proyecto son:

- `cleaned_movies.json`
- `cleaned_ratings.json`
- `train_ratings.json`
- `test_ratings.json`
- `enhanced_movies.json`

A partir de ellos se entrenan o alimentan los distintos recomendadores.

2.2. Limpieza y estructuracion inicial del catalogo

El primer paso consiste en cargar los CSV con el formato correcto de separador y decimales, ya que los datos originales vienen con `;` como delimitador y con coma decimal. Una vez leidos, se aplica una limpieza basica sobre el fichero de ratings: se eliminan todas aquellas valoraciones cuyo `movieId` ya no existe en el catalogo de peliculas disponible. Esta decision es importante porque evita que los recomendadores aprendan sobre items que despues no pueden mostrarse ni recomendarse.

Tras esta depuracion se genera `cleaned_ratings.json`, que actua como version coherente del historial de interacciones. En paralelo se construye `cleaned_movies.json`, una estructura mas manejable que resume por pelicula los campos realmente utiles para el sistema: identificador, `imdb_id`, titulo, `poster_path`, puntuacion media, numero de votos, lista de generos y conjunto de keywords asociadas.

La extraccion de keywords tambien requirio un pequeño preprocesado especifico. En el fichero original las palabras clave aparecen repartidas en varias columnas, de modo que se recorren todas las posiciones relevantes de cada fila, se descartan valores vacios y se construye una lista compacta de keywords por pelicula. Aunque es una transformacion sencilla, tiene bastante impacto posterior porque estas keywords acaban formando parte del espacio textual del recomendador basado en contenido.

2.3. Separacion entre entrenamiento y test

Este apartado se ha decidido mantenerlo breve, puesto que es una fase común de todos los trabajos que hacen los distintos grupos de la asignatura, pero aun así conviene resumirla para contextualizar la evaluación posterior. A partir de los ratings ya limpios, se realiza un *split* por usuario del 70 % para entrenamiento y del 30 % para test. La división no se hace de forma global, sino usuario a usuario, para garantizar que cada perfil conserve información en entrenamiento y que la evaluación se realice sobre películas realmente conocidas por ese mismo usuario en el conjunto de prueba.

Cuando un usuario solo tiene una valoración, esa valoración se deja en entrenamiento para no crear perfiles vacíos. Además, antes de dividir, los ratings de cada usuario se barajan con una semilla fija, de forma que el proceso sea reproducible. El objetivo principal de esta separación es evitar fuga de información: todo lo que se usa para construir perfiles, vecinos y recomendaciones procede exclusivamente del conjunto de entrenamiento, mientras que el conjunto de test queda reservado para medir el rendimiento de los recomendadores.

2.4. Enriquecimiento del catalogo con TMDb

La parte de preprocesado en la que más valor añadido aporta nuestro trabajo es el enriquecimiento del catálogo de películas. Aunque el dataset original ya incluye información mínima para poder trabajar, en muchos casos se queda corto para un recomendador basado en contenido que necesite descripciones, señales semánticas y una presentación visual más completa en la interfaz. Por ello se desarrolló un script específico que toma como entrada `cleaned_movies.json` y lo amplía mediante consultas a la API de TMDb.

Es importante precisar que, aunque el dataset conserva el campo `imdb_id`, el enriquecimiento no se ha realizado directamente contra IMDb, sino contra TMDb. Esto tiene dos ventajas prácticas en nuestro proyecto. La primera es que TMDb expone de forma directa un conjunto amplio de metadatos estructurados por película. La segunda es que facilita el tratamiento de imágenes, ya que proporciona el `poster_path` y el `backdrop_path` oficiales, a partir de los cuales el frontend puede reconstruir la URL completa de la caratula usando la base `https://image.tmdb.org/t/p/original`.

Desde el punto de vista de implementación, este enriquecimiento no se hizo de forma secuencial simple, sino mediante un pipeline asíncrono por lotes. El script procesa lotes de películas y lanza múltiples peticiones concurrentes con `aiohttp`, limitando la concurrencia con un semáforo para no sobrepasar el ritmo permitido por la API. Además, incorpora gestión de *rate limit*, archivo de progreso y copias de seguridad parciales. Esto último es relevante porque el catálogo es grande y una interrupción en mitad del proceso no obligaba a reiniciar todo el enriquecimiento desde cero.

Para cada película se consulta TMDb a partir de su identificador y se fusionan los

nuevos datos con la informacion que ya existia en el JSON limpio. Si una pelicula no aparece en TMDb o la peticion falla, se conservan los datos anteriores y simplemente se rellenan los campos nuevos con valores nulos. Si la consulta tiene exito, se actualizan campos ya existentes como `poster_path`, `puntuacion_media` y `votos`, y ademas se anaden otros muchos atributos adicionales.

Entre los campos nuevos incorporados destacan:

- `overview` y `tagline`
- `runtime` y `release_date`
- `original_title` y `original_language`
- `homepage`, `status` y `popularity`
- `budget` y `revenue`
- `belongs_to_collection` y `adult`
- `production_companies`, `production_countries` y `spoken_languages`
- `vote_average_tmdb` y `vote_count_tmdb` como copia explicita de los valores devueltos por TMDb

Este enriquecimiento tiene dos efectos claros sobre el sistema. El primero es visual: mejora la presentacion de las peliculas porque hace mas fiable la recuperacion de la imagen correcta del poster. El segundo es algorítmico: amplía mucho el volumen de informacion disponible para el recomendador basado en contenido, que pasa a trabajar no solo con generos y keywords, sino tambien con `overview`, `tagline` y otros metadatos semanticos que ayudan a diferenciar mejor unas peliculas de otras.

2.5. Inicializacion y gestion del registro de usuarios

Una vez preparado el catalogo, el siguiente paso es construir el registro persistente de usuarios, almacenado en `Data/user_registry.json`. Este fichero es una pieza central del proyecto porque actua como punto comun entre los tres recomendadores: desde aqui se leen las peliculas vistas, las preferencias declaradas, el vector de generos y, en el caso del colaborativo, la lista de vecinos precalculados.

La poblacion inicial del registro se realiza a partir de `train_ratings.json`, nunca a partir del conjunto completo de ratings. Esta decision vuelve a estar motivada por la necesidad de evitar fuga de informacion. El script de inicializacion agrupa los ratings de entrenamiento por usuario y, para cada uno, crea una entrada con su identificador, estado, tipo de usuario, peliculas vistas y estadisticas basicas como numero de valoraciones y

media de rating. Los usuarios que provienen del dataset se etiquetan como **existing**, mientras que los que se creen posteriormente desde la aplicacion se marcaran como **new**.

Sobre ese registro inicial se ejecuta un segundo proceso muy importante: la construccion del **genre_vector** de cada usuario. Este vector no se obtiene simplemente contando cuantas peliculas de cada genero ha visto el usuario, sino mediante una acumulacion ponderada centrada en sus ratings. Para cada usuario se calcula primero su media de valoracion. Despues, cada rating individual se transforma en un peso relativo:

$$weight = rating - media_usuario$$

Si una pelicula fue puntuada por encima de la media personal del usuario, su contribucion es positiva; si fue puntuada por debajo, su contribucion es negativa. Ese peso se reparte entre los generos de la pelicula y se acumula por genero a lo largo de todo el historial de entrenamiento. Finalmente, las afinidades resultantes se normalizan por usuario al rango $[0, 100]$, de forma que 100 representa el genero de mayor afinidad relativa, 0 los generos no vistos o de afinidad minima, y valores intermedios describen el resto del perfil. A partir de este vector tambien se derivan los **favorite_genres** del usuario, tomando los generos con mayor puntuacion.

Esta forma de construir el perfil tiene una ventaja importante frente a un conteo binario simple: no solo refleja que generos ha consumido un usuario, sino como los ha valorado respecto a su propio criterio. Ademias, el mismo **genre_vector** sirve despues para dos objetivos diferentes: el recomendador basado en contenido lo reduce a sus preferencias mas representativas, mientras que el colaborativo lo utiliza casi completo para comparar usuarios mediante Pearson.

2.6. Creacion de nuevos usuarios y actualizacion del perfil

La gestion de usuarios no termina con los perfiles importados del dataset. La aplicacion permite registrar usuarios nuevos desde la interfaz, introduciendo opcionalmente un nombre y una lista inicial de generos favoritos. En el momento del alta se genera un nuevo identificador, se crea una entrada en **user_registry.json** y se inicializa su estructura de preferencias con una lista vacia de peliculas vistas y un **genre_vector** basico.

Ese vector inicial se construye de forma sencilla: cada genero favorito seleccionado en el registro recibe una afinidad de 100. Esta inicializacion es deliberadamente simple, pero suficiente para que el sistema pueda empezar a recomendar incluso antes de que el usuario haya puntuado peliculas. En otras palabras, permite un manejo razonable del problema de *cold start* para usuarios recién creados.

Cuando un usuario nuevo, o incluso uno ya existente, puntua una pelicula desde la aplicacion, el sistema actualiza varios elementos a la vez. En primer lugar, inserta o modifica la valoracion en **train_ratings.json**. En segundo lugar, anade la pelicula a

`watched_movies` si todavia no estaba presente. En tercer lugar, recalcula su media de valoracion y ajusta el `genre_vector` usando los generos de la pelicula puntuada. Este ajuste incremental se realiza mediante una heuristica sencilla: los generos de la pelicula reciben un incremento o decremento proporcional a `rating - 2.5`, y el resultado se recorta al rango $[0, 100]$.

Por ultimo, cuando cambia el perfil de un usuario se invalidan sus vecinos colaborativos y, si existe, la matriz cacheada de preferencias del recomendador colaborativo. Esto obliga a recalcular la informacion de similitud en la siguiente ejecucion y evita que el sistema siga utilizando relaciones de vecindad desactualizadas. Ademas, el colaborativo incorpora un mecanismo de respaldo: si detecta usuarios sin `genre_vector` util, intenta derivarlo automaticamente a partir de sus generos favoritos y de las peliculas vistas, para no dejar perfiles vacios.

2.7. Papel de esta fase en el resto del sistema

Toda esta etapa de preprocesado condiciona directamente la calidad del recomendador final. El basado en contenido depende del enriquecimiento de peliculas para disponer de senales textuales y semanticas suficientes. El colaborativo depende de que el registro de usuarios y el `genre_vector` esten bien contruidos para encontrar vecinos fiables. El hibrido, por su parte, hereda tanto las fortalezas como las limitaciones de ambos. Por eso, aunque el preprocesado pueda parecer una fase previa a los algoritmos, en realidad forma parte esencial del comportamiento final del sistema.

3. Sistema recomendador basado en contenido

Este apartado explica como se recomienda una pelicula a partir del perfil de gustos del usuario, de las caracteristicas de las peliculas y otras características. A continuación se define el calculo del *rating* del recomendador basado en contenido para que sea coherente con una formulacion interpretable y estable. El objetivo es priorizar la personalizacion del usuario, sin perder senales de calidad global, fiabilidad estadistica de cada pelicula y una medida semantica adicional basada en el contenido textual.

3.1. Funcion de rating implementada

La puntuacion final estimada para un usuario u y una pelicula i se define como:

$$r_{ui} = \alpha \cdot Score_{contenido}(u, i) + \beta \cdot Score_{calidad}(i) + \gamma \cdot Score_{popularidad}(i) + \delta \cdot Score_{similitud}(u, i)$$

con los pesos:

$$\alpha = 0,50, \quad \beta = 0,20, \quad \gamma = 0,15, \quad \delta = 0,15$$

Esta formulacion mantiene el peso principal en las preferencias del usuario, incorporando adicionalmente una componente semantica que permite capturar similitudes mas finas entre peliculas.

3.2. Componentes del modelo

3.2.1. Score de contenido

Se calcula a partir de la coincidencia entre generos de la pelicula y preferencias de genero del usuario (normalizadas). Para cada pelicula se obtiene una afinidad en $[0, 1]$, ponderando la intensidad de preferencia y dividiendo por el numero de generos del item para evitar sesgo hacia peliculas con muchas etiquetas.

3.2.2. Score de calidad

Se normaliza la valoracion media de la pelicula a $[0, 1]$:

$$Score_{calidad} = \frac{vote_average}{10}$$

Cuando no existe esta metrica, se aplica *fallback* usando medias historicas de ratings disponibles.

3.2.3. Score de popularidad

Se modela la fiabilidad de la valoracion mediante una normalizacion logaritmica del numero de votos:

$$Score_{popularidad} = \frac{\log(1 + votos)}{\log(1 + votos_{max})}$$

Si no hay `vote_count`, se usa como *fallback* el numero de valoraciones historicas del dataset de entrenamiento.

3.2.4. Score de similitud semantica

Se introduce una componente adicional basada en la similitud del coseno entre representaciones vectoriales del contenido textual (*overview*) de las peliculas. En lugar de comparar directamente peliculas entre si, se construye un perfil textual del usuario agregando los *overviews* de las peliculas mejor valoradas por dicho usuario.

Formalmente:

$$Score_{similitud}(u, i) = \cos(\vec{profile}_u, \vec{overview}_i)$$

donde:

- $\vec{overview}_i$ es la representacion vectorial del *overview* de la pelicula i .
- $\vec{profile}_u$ es el perfil textual del usuario, obtenido como combinacion ponderada de los *overviews* de las peliculas previamente valoradas positivamente.

Esta componente permite capturar similitudes semanticas que no quedan reflejadas unicamente mediante generos, refinando la calidad de la recomendacion. Su peso se mantiene moderado para no desplazar las senales principales del modelo.

3.3. Cambios de implementacion realizados

Se aplican los siguientes cambios principales en el modulo `content_based.py`:

- Se incorpora el nuevo parametro **delta** asociado a la similitud semantica.
- Se sustituye la combinacion anterior basada unicamente en senales estructuradas por una combinacion de **contenido + calidad + popularidad + similitud textual**.
- Se implementa el calculo de afinidad por generos con caches auxiliares para mejorar rendimiento.
- Se anade la generacion del perfil textual del usuario a partir de *overviews* previamente valorados.
- Se integra el calculo de similitud coseno sobre representaciones vectoriales.
- Se adapta el ranking inicial de candidatos para priorizar la afinidad de contenido.
- Se mantiene la normalizacion final del score para presentacion y ordenacion estable.

3.4. Explicabilidad y trazabilidad

Ademas del calculo, se rediseña la explicacion de recomendaciones para que refleje exactamente la formula del modelo. Para cada pelicula recomendada se muestra:

- la combinacion ponderada aplicada $(\alpha, \beta, \gamma, \delta)$,
- el desglose de los cuatro componentes (*contenido*, *calidad*, *popularidad*, *similitud*),
- una justificacion semantica basada en coincidencia de generos y proximidad textual con el perfil del usuario.

En la interfaz, tambien se ha añadido este desglose en las tarjetas de recomendacion del modo basado en contenido, mejorando la interpretabilidad para el usuario final y facilitando la validacion del comportamiento del sistema.

4. Sistema recomendador colaborativo

El sistema recomendador colaborativo se basa en una idea distinta al recomendador basado en contenido. Aquí no se comparan películas entre sí, sino usuarios entre sí. El objetivo es detectar que usuarios tienen perfiles de gusto parecidos y reutilizar las valoraciones de esos usuarios similares para recomendar películas al usuario objetivo. Por tanto, el algoritmo responde a la pregunta: si otros usuarios con gustos parecidos han puntuado bien una película que yo todavía no he visto, hasta qué punto es razonable pensar que esa película también puede interesarme.

4.1. Datos de entrada y papel de cada fuente

La implementación colaborativa se apoya sobre tres fuentes principales:

- `user_registry.json`, donde se almacenan los usuarios activos, su `genre_vector`, sus películas vistas y, en muchos casos, los vecinos ya precalculados.
- `train_ratings.json`, que contiene las valoraciones históricas utilizadas tanto para construir candidatos como para calcular la predicción final de rating.
- `enhanced_movies.json`, que en este modelo no se usa para medir similitud entre usuarios, pero sí sirve para recuperar información de las películas y para derivar géneros cuando un usuario no tiene todavía un vector de preferencias completo.

Además, el sistema guarda un artefacto auxiliar muy importante: la matriz de preferencias usuario-género se cachea en un fichero `npz`. Esto evita reconstruir toda la estructura en cada ejecución y hace viable reutilizar el colaborativo en la aplicación sin recalcular desde cero la comparación entre miles de pares de usuarios.

4.2. Representación del usuario

La decisión de modelado más importante de este recomendador es que cada usuario se representa mediante un vector de afinidad por género, y no mediante una matriz usuario-item clásica. Es decir, cada usuario queda descrito por una estructura del tipo:

$$\vec{p}_u = [p_{u1}, p_{u2}, \dots, p_{un}]$$

donde cada componente indica cuánto le interesa al usuario un género concreto en una escala normalizada entre 0 y 100. Este vector procede del preprocesado descrito anteriormente y resume el comportamiento histórico del usuario de forma mucho más compacta que una matriz con miles de películas.

Esta elección está justificada por dos razones. La primera es que el catálogo de películas es muy grande y una matriz usuario-item sería mucho más dispersa. La segunda es que,

para buscar vecinos, nos interesa una representacion que concentre la senal de preferencia y reduzca el problema de falta de solapamiento entre usuarios. En lugar de comparar si dos personas han puntuado exactamente las mismas peliculas, comparamos si muestran patrones similares de afinidad hacia los generos.

4.3. Construccion de la matriz usuario-genero

A partir de esos vectores individuales se construye una matriz usuario-genero en la que cada fila representa un usuario activo y cada columna representa un genero. Para que la matriz sea consistente, primero se fija un orden estable de generos; despues, para cada usuario, se genera una fila numerica siguiendo exactamente ese mismo orden. Si un genero no aparece en el perfil del usuario, su valor se rellena con 0.

Formalmente, si hay m usuarios activos y n generos, la matriz resultante puede verse como:

$$M \in \mathbb{R}^{m \times n}$$

El sistema mantiene tambien dos mapeos auxiliares para poder pasar de identificadores logicos de usuario a indices de fila y viceversa:

- `user_id_to_idx`
- `idx_to_user_id`

Una vez creada, la matriz se guarda en disco como `csr_matrix`. El uso de un formato disperso no cambia el significado del algoritmo, pero mejora la gestion de memoria y la reutilizacion posterior.

Hay ademas un detalle importante de robustez: antes de construir la matriz, el colaborativo comprueba si existen usuarios activos cuyo `genre_vector` este vacio o no tenga valores positivos. Si eso ocurre, intenta derivar un perfil minimo a partir de los generos favoritos declarados y de las peliculas vistas. De esta forma se evita dejar usuarios inutilizables dentro del sistema y se reduce el numero de casos en los que la recomendacion colaborativa no puede arrancar.

4.4. Calculo de similitud entre usuarios

La similitud entre usuarios se calcula con el coeficiente de correlacion de Pearson. Esta medida no se limita a comprobar si dos usuarios tienen valores altos en las mismas dimensiones, sino que capta si la forma general de sus preferencias evoluciona de manera parecida. Por eso es una opcion adecuada cuando queremos comparar perfiles relativos y no solo magnitudes absolutas.

Dados dos vectores de preferencias $\vec{p}_u$ y $\vec{p}_v$, la similitud se calcula como:

$$sim(u, v) = \frac{\sum_k (p_{uk} - \bar{p}_u)(p_{vk} - \bar{p}_v)}{\sqrt{\sum_k (p_{uk} - \bar{p}_u)^2} \cdot \sqrt{\sum_k (p_{vk} - \bar{p}_v)^2}}$$

El resultado queda en el rango $[-1, 1]$. En esta implementacion solo se conservan vecinos con similitud positiva. Si el denominador es 0, la similitud se fuerza a 0 para evitar resultados inestables o indefinidos.

Uno de los puntos metodologicos mas importantes del trabajo es decidir sobre que parte del vector se calcula Pearson. En nuestro caso no se usa siempre la misma regla. Si dos usuarios comparten suficientes generos con valor no nulo, se calcula la correlacion sobre esa interseccion de preferencias realmente expresadas. Si el numero de preferencias comunes no alcanza el umbral fijado, se utiliza la union completa del vector, es decir, todo el espacio de generos con ceros incluidos. En el codigo actual, el umbral minimo se fija en:

$$MIN_COMMON_PREFS = 10$$

Esta estrategia hibrida entre interseccion y union esta alineada con las transparencias de la asignatura y responde a un compromiso razonable. Cuando hay bastante informacion comun, la interseccion es mas informativa porque compara gustos efectivamente compartidos. Cuando el solapamiento es pequeno, limitarse a unos pocos generos puede producir correlaciones artificiales; en esos casos es preferible usar mas contexto y hacer la comparacion mas conservadora.

4.5. Obtencion y persistencia de vecinos

Una vez calculada la similitud del usuario objetivo frente al resto de usuarios activos, se descartan el propio usuario y todas las correlaciones no positivas. Los candidatos restantes se ordenan de mayor a menor similitud y se conservan como maximo los N mejores vecinos. En la implementacion actual ese maximo es:

$$top_n = 40$$

La eleccion de un maximo de 40 vecinos busca equilibrar calidad y coste. Si se incluyen demasiados vecinos, acaban entrando usuarios poco parecidos que introducen ruido en la prediccion. Si se usan muy pocos, la recomendacion puede quedar dominada por una evidencia demasiado debil. Por eso se adopta una lista acotada pero suficientemente amplia.

Es importante comentar en la memoria que la correlacion de Pearson es simetrica, pero la relacion de vecindad no tiene por que serlo. Dos usuarios pueden tener la misma similitud numerica, pero si uno entra en el top-40 del otro y no al reves, la relacion final de vecindad sera asimetrica. El sistema permite ademas precalcular esta informacion para

todos los usuarios y guardarla en el registro, de modo que la aplicacion pueda reutilizarla sin repetir el proceso en cada recomendacion.

4.6. Generacion de peliculas candidatas

Una vez obtenida la lista de vecinos, el siguiente paso es generar el conjunto de peliculas candidatas. En este modelo una pelicula candidata es una pelicula que cumple dos condiciones: ha sido puntuada favorablemente por al menos un vecino y no figura entre las peliculas ya vistas por el usuario objetivo. La nocion de puntuacion favorable tambien esta parametrizada en el codigo, y actualmente se considera favorable cualquier rating mayor o igual que:

$$FAVORABLE_RATING_THRESHOLD = 3,5$$

El proceso es sencillo pero importante: para cada vecino se recorren sus ratings de entrenamiento, se seleccionan solo las peliculas bien valoradas y se filtran aquellas que el usuario ya tenga en `watched_movies`. El uso de un conjunto elimina automaticamente duplicados, de forma que si varios vecinos han visto una misma pelicula, esta pelicula sigue apareciendo una sola vez en la lista de candidatos.

4.7. Prediccion del rating de una pelicula

Una vez contruidos los candidatos, el sistema estima un rating predicho para cada pelicula. Para ello se buscan los vecinos que realmente contribuyen a ese item concreto, es decir, vecinos que lo hayan puntuado en `train_ratings.json`. Cada contribuidor aporta dos datos: su similitud con el usuario objetivo y la puntuacion que dio a la pelicula.

La prediccion principal se calcula mediante una media ponderada por similitud:

$$\hat{r}_{ui} = \frac{\sum_{v \in V_u(i)} sim(u, v) r_{vi}}{\sum_{v \in V_u(i)} |sim(u, v)|}$$

donde $V_u(i)$ es el subconjunto de vecinos de u que han puntuado la pelicula i . Esta formula hace que las valoraciones de vecinos mas parecidos tengan mas peso en el resultado final. Si por algun motivo el denominador resulta 0, la prediccion se fija a 0.

4.8. Confianza y score final de ranking

Una diferencia importante respecto a una implementacion colaborativa basica es que aqui no se usa solo el rating predicho para ordenar peliculas. El sistema calcula tambien una medida de confianza. La confianza depende de dos elementos: la similitud media de los vecinos que contribuyen a la pelicula y el soporte de la recomendacion, es decir, cuantos vecinos participan realmente en esa prediccion.

Para medir soporte se utiliza una referencia fija de tres contribuyentes:

$$CONFIDENCE_NEIGHBOR_TARGET = 3$$

Con ello se define un `support_ratio` acotado en $[0, 1]$:

$$support_ratio(u, i) = \min \left(1, \frac{num_contributors(u, i)}{3} \right)$$

La confianza final se calcula como:

$$confidence(u, i) = mean_similarity(u, i) \cdot support_ratio(u, i)$$

Y el score utilizado para ordenar candidatos queda definido como:

$$ranking_score(u, i) = \hat{r}_{ui} \cdot confidence(u, i)$$

Esta parte es clave para justificar el comportamiento del algoritmo. Dos películas pueden tener un rating predicho parecido, pero no ofrecer el mismo grado de fiabilidad. Una recomendación sostenida por varios vecinos muy similares es más creíble que otra basada en un único vecino o en similitudes muy bajas. Por eso separar predicción y confianza mejora el ranking final.

4.9. Ordenación final y control de soporte

Una vez calculadas las predicciones, el sistema no ordena todas las películas por igual. Primero separa las recomendaciones fuertes de las débiles. Se considera fuerte una recomendación respaldada por al menos:

$$MIN_RECOMMENDATION_CONTRIBUTORS = 2$$

Las películas con soporte suficiente se colocan delante de las que solo tienen un contribuidor. Dentro de cada grupo, la ordenación se hace por `ranking_score`, después por confianza, después por rating predicho, similitud media y número de contribuidores. Esta decisión evita que una película con nota muy alta pero respaldada por un único vecino domine injustificadamente la lista final.

El resultado que devuelve el recomendador puede ser simple, si solo queremos la película y su rating predicho, o detallado, si se desea información adicional para depuración o explicación. En este segundo caso se exponen también el número de contribuidores, la similitud media, la confianza y el `ranking_score`.

4.10. Casos de usuarios nuevos y perfiles incompletos

El colaborativo es el modelo mas sensible al problema de *cold start*. Si un usuario nuevo no tiene apenas historial y tampoco dispone de un `genre_vector` util, la busqueda de vecinos puede ser muy pobre o incluso imposible. Para reducir ese problema, el sistema intenta reconstruir un perfil inicial a partir de `favorite_genres` y de las peliculas vistas. Los generos favoritos reciben un peso mayor que las peliculas vistas, y el resultado se normaliza al rango $[0, 100]$.

Este mecanismo no sustituye a un perfil aprendido con ratings reales, pero permite que el sistema tenga una base minima sobre la que buscar vecinos. Aun asi, si el usuario no ofrece suficiente informacion o si no se encuentran vecinos con correlacion positiva, el recomendador colaborativo simplemente no genera recomendaciones utiles. Esta limitacion es inherente al enfoque.

4.11. Decisiones de implementacion que interesa destacar

- El algoritmo compara usuarios por preferencias de genero y no por coincidencia directa en items, para reducir dispersion y hacer mas estable la similitud.
- Se usa Pearson porque capta la forma relativa del perfil y no solo la magnitud de las puntuaciones.
- La comparacion mezcla interseccion y union de preferencias segun el solapamiento disponible, con un umbral de 10 preferencias comunes.
- La recomendacion no se ordena solo por rating predicho; tambien incorpora confianza y soporte vecinal.
- Los vecinos pueden persistirse en el registro de usuarios para ahorrar coste en tiempo de ejecucion.

4.12. Limitaciones a comentar

- La calidad del sistema depende mucho de la calidad del `genre_vector` construido en el preprocesado.
- Si los usuarios tienen perfiles pobres o demasiado dispersos, pueden aparecer falsos vecinos o no encontrarse vecinos utiles.
- El modelo sigue siendo sensible al *sparsity*: aunque se reduzca el espacio a generos, no todos los usuarios ofrecen la misma cantidad de informacion.
- La prediccion final se apoya en heurísticas de confianza y umbrales fijos, que podrian ajustarse mejor mediante evaluacion experimental.

5. Sistema recomendador hibrido

El sistema hibrido debe presentarse como una combinacion ponderada de los dos enfoques anteriores. El objetivo de esta seccion es explicar como se mezclan las listas y por que los pesos no son fijos.

5.1. Idea general

La recomendacion hibrida sigue tres pasos:

- Obtener una lista de candidatos del recomendador basado en contenido.
- Obtener una lista de candidatos del recomendador colaborativo.
- Unificar ambas listas y recalcular una puntuacion comun para ordenar los items finales.

5.2. Normalizacion de puntuaciones

Uno de los puntos que hay que explicar mejor en la memoria es que ambos modelos no producen ratios en la misma escala. Por eso, antes de mezclar:

- El score de contenido se interpreta ya en el rango $[0, 1]$.
- El score colaborativo se normaliza a partir de `ranking_score` para llevarlo tambien a $[0, 1]$.

Esta justificacion es importante porque evita una mezcla engañosa en la que un modelo domine al otro por pura escala numerica.

5.3. Pesos dinamicos

La mezcla final se expresa como:

$$score_{hibrido}(u, i) = \alpha_u \cdot score_{contenido}(u, i) + \beta_u \cdot score_{colaborativo}(u, i)$$

con la restriccion:

$$\alpha_u + \beta_u = 1$$

La memoria debe remarcar que α y β no son constantes. Se calculan en cada recomendacion a partir de la informacion disponible para ese usuario:

- La senal de contenido depende de la calidad del perfil de generos, del historial y del numero de candidatos que el modelo es capaz de generar.

- La señal colaborativa depende del número de vecinos, de su similitud media y del soporte de candidatos.
- Antes de normalizar, ambas señales se suavizan para que el híbrido no se convierta en un sistema casi monolítico cuando las dos fuentes aportan información útil.

5.4. Que debe explicarse en la mezcla final

- Como se eliminan ítems duplicados entre ambas listas.
- Como se combinan las contribuciones de contenido y colaborativo para un mismo ítem.
- Por qué se usa un conjunto de candidatos amplio antes de quedarse con el top final.
- Como se generan explicaciones del resultado híbrido indicando el peso relativo de cada enfoque.

5.5. Decisiones de implementación que interesa justificar

- Usar pesos dinámicos en lugar de fijar una mezcla constante 50/50.
- Normalizar las escalas antes de combinar scores.
- Aprovechar la robustez del contenido y la capacidad de descubrimiento del colaborativo.

5.6. Limitaciones a comentar

- Los pesos siguen siendo heurísticas, no parámetros aprendidos automáticamente.
- El modelo depende de que ambos recomendadores base funcionen de forma razonable.
- Su calibración ideal debería validarse con evaluación offline.

6. Sistema recomendador para grupos

Aunque esta parte todavía no esté implementada, es conveniente reservar ya una sección específica en la memoria para dejar claro como se planteará el diseño. El objetivo de este bloque es mostrar la lógica prevista y las decisiones que habrá que cerrar en la siguiente iteración.

6.1. Enfoque propuesto

La idea general es seguir un esquema de agregacion de items:

- Seleccionar los usuarios que forman el grupo.
- Elegir una unica tecnica de recomendacion para todos los miembros del grupo: contenido, colaborativa o hibrida.
- Obtener una recomendacion individual para cada usuario.
- Eliminar las peliculas que aparezcan en el historico de cualquier miembro del grupo.
- Agregar las listas individuales en una lista comun para el grupo.
- Calcular un ratio final de interes del grupo para cada item.

6.2. Que debe explicarse en esta seccion

- Que tecnica de agregacion se elige finalmente y por que.
- Cuantos usuarios como maximo se permiten por grupo.
- Como se combinan items repetidos entre distintas listas individuales.
- Que ocurre cuando un item gusta mucho a unos miembros y poco a otros.
- Como se interpretara el score grupal en la interfaz.

6.3. Decision de diseno pendiente

Como primer planteamiento razonable, la memoria puede dejar planteado que se compararan tecnicas de agregacion sencillas y explicables, por ejemplo:

- *Average*, si se quiere priorizar el consenso medio.
- *Borda count*, si se quiere premiar la presencia recurrente de un item en varias listas.
- *Least misery*, si se quiere evitar recomendaciones muy malas para algun miembro del grupo.

Aunque todavia no haya codigo, incluir este apartado en la memoria es util porque muestra continuidad del trabajo y deja definida la extension natural del sistema.

7. Interfaz y flujo de uso

Esta seccion puede ser breve, pero ayuda a conectar los algoritmos con la aplicacion final. La memoria debe indicar que la interfaz permite seleccionar el tipo de recomendador y mostrar las recomendaciones resultantes al usuario.

7.1. Que conviene explicar

- Que el frontend permite elegir entre contenido, colaborativo e hibrido.
- Que las recomendaciones se presentan sobre peliculas no vistas por el usuario.
- Que el sistema puede mostrar informacion adicional para ayudar a interpretar la recomendacion.
- Que el futuro recomendador de grupos deberia integrarse en este mismo flujo de seleccion.

8. Evaluacion del sistema

Aunque la evaluacion todavia no este cerrada, la memoria deberia reservar ya este apartado porque forma parte natural del trabajo y permite comparar tecnicas de forma objetiva.

8.1. Protocolo de evaluacion que debe describirse

- Usar usuarios del dataset, no usuarios sinteticos.
- Seleccionar un conjunto fijo de usuarios para comparar todos los recomendadores en igualdad de condiciones.
- Generar recomendaciones con el conjunto de entrenamiento y contrastarlas con el conjunto de test.
- Eliminar de la comparacion peliculas que ya no existan en el catalogo procesado.

8.2. Metricas que interesa incluir

- Precision.
- Recall.
- F1.
- MAE.

- nDCG.

8.3. Que debe contarse en la memoria

- Que mide cada metrica y por que es util.
- Como se construyen las listas comparadas para cada metrica.
- Como se comparan entre si contenido, colaborativo e hibrido.
- Que sistema funciona mejor segun acierto, cobertura y calidad del ranking.

9. Conclusiones y trabajo pendiente

La memoria deberia cerrarse con un apartado muy sintetico de conclusiones. No hace falta repetir toda la teoria; basta con resumir que aporta cada recomendador y que compromisos presenta.

Aqui conviene dejar tres ideas claras:

- El recomendador basado en contenido es el mas interpretable y robusto cuando hay pocos usuarios.
- El colaborativo aporta descubrimiento, pero depende mas de la calidad del perfil y de los vecinos disponibles.
- El hibrido busca equilibrar ambos enfoques y es el candidato natural a mejor recomendador general, aunque todavia necesite calibracion mediante evaluacion.

Por ultimo, esta seccion debe mencionar de forma explicita el trabajo pendiente:

- Implementar el recomendador para grupos.
- Completar la evaluacion experimental offline.
- Ajustar pesos y umbrales a partir de resultados medidos.